

CLI Lab (CodeGrade)

Learning Goals

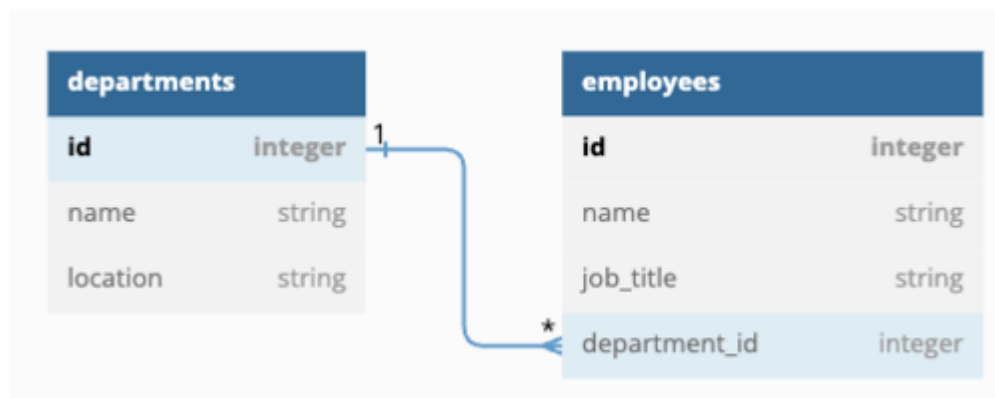
- Implement a CLI for an ORM application

Instructions

This is **not** a test-driven lab. You will test your code using a command line interface (CLI).

Run `pipenv install` to create your virtual environment and `pipenv shell` to enter the virtual environment.

We'll continue to add a command line interface to the company ORM application from the previous lesson:



The directory structure is as follows:

```
.
├── lib
│   ├── models
│   │   ├── __init__.py
│   │   ├── department.py
│   │   └── employee.py
│   ├── testing
│   │   ├── conftest.py
│   │   ├── department_orm_test.py
│   │   ├── department_property_test.py
│   │   ├── employee_orm_test.py
│   │   └── employee_property_test.py
│   ├── cli.py
│   ├── company.db
│   ├── debug.py
│   ├── helpers.py
│   └── seed.py
├── Pipfile
├── Pipfile.lock
├── pytest.ini
└── README.md
```

Seeding the database with sample data

The file `lib/seed.py` contains code to initialize the database with sample departments and employees. Run the following command to seed the database:

```
python lib/seed.py
```

You can use the SQLITE EXPLORER extension to explore the initial database contents. (Another alternative is to run `python lib/debug.py` and use the `ipdb` session to explore the database)

cli.py and helpers.py

The file `lib/cli.py` contains a command line interface for our company database application. The CLI displays a menu of commands. Each numeric choice will call a function in `lib/helpers.py`. The starter code implements options `0` through `6`, calling ORM methods related to the `Department` class.

Run `python lib/cli.py` and confirm options 0 through 6 work.

Please `select` an option:

- 0. Exit the program
- 1. List all departments
- 2. Find department by name
- 3. Find department by id
- 4: Create department
- 5: Update department
- 6: Delete department
- 7. List all employees
- 8. Find employee by name
- 9. Find employee by id
- 10: Create employee
- 11: Update employee
- 12: Delete employee
- 13: List all employees `in` a department

You will implement the helper functions related to the `Employee` class (options `7` through `13`). The file `lib/helpers.py` has a function for each option containing a `pass` statement. You need to update each function to replace the `pass` statement with code to implement the necessary functionality.

Each helper function should call ORM methods in the `Employee` and `Department` classes.

list_employees()

Implement the `list_departments()` function in `lib/helpers.py` . The function should get all employees stored in the database, then print each employee on a new line.

Test the function by selecting option `7` when you run `python lib/cli.py` :

Please `select` an option:

```
....
> 7
<Employee 1: Amir, Accountant, Department ID: 1>
<Employee 2: Bola, Manager, Department ID: 1>
<Employee 3: Charlie, Manager, Department ID: 2>
<Employee 4: Dani, Benefits Coordinator, Department ID: 2>
<Employee 5: Hao, New Hires Coordinator, Department ID: 2>
```

`find_employee_by_name()`

The function `find_employee_by_name()` should prompt for a `name` and then find the `Employee` instance with that name and print their information, or print an error message if the employee does not exist.

Test the function by selecting option `8` when you run `python lib/cli.py` .

Try entering a name that exists in the database:

```
> 8
Enter the employee's name: Dani
<Employee 4: Dani, Benefits Coordinator, Department ID: 2>
```

Try entering a name not in the database:

```
> 8
Enter the employee's name: Fred
Employee Fred not found
```

find_employee_by_id()

The function `find_employee_by_id()` should prompt for an id and then find the `Employee` instance with that id and print their information, or print an error message if the employee does not exist.

Test the function by selecting option `9` when you run `python lib/cli.py`.

Try entering an id that exists in the database:

```
> 9
Enter the employee's id: 2
<Employee 2: Bola, Manager, Department ID: 1>
```

Try entering an id not in the database:

```
> 9
Enter the employee's id: 99
Employee 99 not found
```

create_employee()

The function `create_employee()` should:

1. Prompt for and read in a name, job title, and department id.
2. Create and persist a new `Employee` class instance, surrounding the code in a `try/except` block in case an exception is thrown by the `name`, `job_title`, or `department_id` property setter methods.
3. Print a message indicating that the `Employee` object was successfully created, or print an error message if an exception is thrown.

Test the function by selecting option `10` when you run `python lib/cli.py`.

```
> 10
Enter the employee's name: Ira
Enter the employee's job title: Manager
```

Enter the employee's department id:1

Success: <Employee 6: Ira, Manager, Department ID: 1>

Confirm the employee was added to the database by selecting option **7** to list all employees:

> 7

<Employee 1: Amir, Accountant, Department ID: 1>

<Employee 2: Bola, Manager, Department ID: 1>

<Employee 3: Charlie, Manager, Department ID: 2>

<Employee 4: Dani, Benefits Coordinator, Department ID: 2>

<Employee 5: Hao, New Hires Coordinator, Department ID: 2>

<Employee 6: Ira, Manager, Department ID: 1>

Try entering invalid data for name or job title:

> 10

Enter the employee's name:

Enter the employee's job title: Programmer

Enter the employee's department id: 1

Error creating employee: Name must be a non-empty string

Try entering an invalid department id:

> 10

Enter the employee's name: Jani

Enter the employee's job title: Accountant

Enter the employee's department id:99

Error creating employee: department_id must reference a department in the database

update_employee()

The function `update_employee()` should:

1. Prompt for and read in the employee id.
2. Print an error message if the employee is not in the database. If the employee is in the database, attempt to do the following steps within a `try-except` block to catch any exceptions, printing an error message if an exception is thrown.
3. Prompt for a new name to update the `name` attribute (property setter may throw an exception).
4. Prompt for a new job title to update the `job_title` attribute (property setter may throw an exception).
5. Prompt for the employee's new department id to update the `department_id` attribute (property setter may throw an exception).
6. Update the employee in the database.
7. Print a success message after a successful update, or print an appropriate error message if an exception is thrown.

Test the function by selecting option `11` when you run `python lib/cli.py`.

```
> 11
Enter the employee's id: 3
Enter the employees's new name: Charles
Enter the employee's new job title: Director
Enter the employees's new department id: 1
Success: <Employee 3: Charles, Director, Department ID: 1>
```

Confirm the database was updated by listing all employees:

```
> 7
<Employee 1: Amir, Accountant, Department ID: 1>
<Employee 2: Bola, Manager, Department ID: 1>
<Employee 3: Charles, Director, Department ID: 1>
<Employee 4: Dani, Benefits Coordinator, Department ID: 2>
<Employee 5: Hao, New Hires Coordinator, Department ID: 2>
<Employee 6: Ira, Manager, Department ID: 1>
```

Try entering an invalid employee id:

```
> 11
Enter the employee's id: 99
```

Employee 99 not found

Try entering an invalid name:

```
> 11
Enter the employee's id: 4
Enter the employees's new name:
Error updating employee: name must be a non-empty string
```

Try entering an invalid job title:

```
> 11
Enter the employee's id: 4
Enter the employees's new name: Danielle
Enter the employee's new job title:
Error updating employee: job_title must be a non-empty string
```

Try entering an invalid department id:

```
> 11
Enter the employee's id: 4
Enter the employees's new name: Danielle
Enter the employee's new job title: Senior Benefits Coordinator
Enter the employees's new department id: 99
Error updating employee: department_id must reference a department in the database
```

delete_employee()

The function `delete_employee()` should prompt for the employee `id` and delete the employee from the database if it exists and print a confirmation message, or print an error message if the employee is not in the database.

Test the function by selecting option `12` when you run `python lib/cli.py`.


```
> 12
Enter the employee's id: 1
Employee 1 deleted
```

Confirm the employee was deleted by listing all employees:

```
> 7
<Employee 2: Bola, Manager, Department ID: 1>
<Employee 3: Charles, Director, Department ID: 1>
<Employee 4: Dani, Benefits Coordinator, Department ID: 2>
<Employee 5: Hao, New Hires Coordinator, Department ID: 2>
<Employee 6: Ira, Manager, Department ID: 1>
```

Try entering a non-existent employee id:

```
> 12
Enter the employee's id: 99
Employee 99 not found
```

list_department_employees()

You may want to reseed the database to get the same output.

The function `list_department_employees()` should:

1. prompt for a department id.
2. find the department with that id from the database.
3. if the department exists in the database, get the department's employees (HINT: call the `employees()` instance method) and loop to print each employee's data on a separate line.
4. if the department does not exist in the database, print an error message.

Test the function by selecting option **13** when you run `python lib/cli.py` .

```
> 13
```

```
Enter the department's id: 1
```

```
<Employee 1: Amir, Accountant, Department ID: 1>
```

```
<Employee 2: Bola, Manager, Department ID: 1>
```

```
> 13
```

```
Enter the department's id: 2
```

```
<Employee 3: Charlie, Manager, Department ID: 2>
```

```
<Employee 4: Dani, Benefits Coordinator, Department ID: 2>
```

```
<Employee 5: Hao, New Hires Coordinator, Department ID: 2>
```

Try an id that does not match an existing department:

```
> 13
```

```
Enter the department's id: 99
```

```
Department 99 not found
```

Success! Use git to push and submit your lab to Canvas.
